HSA as Sparse Causal Routing

Formalization, Current Packing Strategy, and a Path Toward Typed Semantic Graphs

April 2026

Abstract

This note formalizes the current hierarchical sparse attention (HSA) stack as a decoder-causal sparse routing problem over a sequence graph, explains the current dense HSA reference, schedule-based sparse realization, and packing strategy, and distinguishes two operational HSA regimes that matter for current benchmarking: a naive adjacency-driven sparse realization and a packed HSA path that reuses precomputed cached sidecars. The note records the current isolated speedups of packed HSA over naive HSA, places those results next to flat and sliding-window FA4 baselines, and then sketches a realistic "leapfrog" extension path from binary hierarchical masks toward typed, directed, and eventually signed semantic routing graphs that could support precursors to causal reasoning in autoregressive language models. The central claim is that the current implementation is not "arbitrary graph attention," but it already contains reusable pieces of a graph compiler, sparse runtime, and offline sidecar system that can be repurposed for richer semantic graphs.

1 Motivation

The current HSA recipe uses semantic structure already present in text representation—sentence anchors, section anchors, document anchors, or other hierarchical boundary markers—to build a sparse causal attention graph. This graph is then realized either as a dense semantic mask or as an equivalent schedule-based sparse computation.

The long-range motivation is twofold:

1. reduce attention cost while preserving relevance over long contexts,
2. bias the model toward structured information flow rather than letting dense attention allocate probability mass over all causal predecessors.

That is already useful for "context rot" mitigation, but it also opens a more ambitious direction: moving from document hierarchy to typed semantic graphs and eventually to typed directional message passing that can approximate parts of causal reasoning without claiming true interventional semantics.

2 Formalizing Current HSA

2.1 Sequence, graph, and support mask

Let a decoder consume tokens $x_1, \ldots, x_T$. Standard causal self-attention defines a dense lower-triangular support:

$$\mathcal{E}_{\text{causal}} = \{(i, j) \mid 1 \le j \le i \le T\}.$$

Current HSA replaces this with a sparse causal subgraph

$$G = (V, E), \quad V = \{1, \dots, T\}, \quad E \subseteq \mathcal{E}_{\text{causal}}.$$

Equivalently, it defines a binary support mask

$$S_{ij} = \begin{cases} 1 & \text{if } (i,j) \in E, \\ 0 & \text{otherwise.} \end{cases}$$

The graph E is not learned online. It is compiled from token-side semantic metadata such as `keep_ids` and `hash_ids`, which encode the hierarchical structure of the document.

2.2 Dense HSA reference

Current dense HSA is a masked exact attention reference:

$$\text{score}_{ij} = \frac{q_i^\top k_j}{\sqrt{d}} + m_{ij},$$

where

$$m_{ij} = \begin{cases} 0 & \text{if } S_{ij} = 1, \\ -\infty & \text{if } S_{ij} = 0. \end{cases}$$

Then

$$\alpha_{ij} = \frac{\exp(\text{score}_{ij})}{\sum_{\ell : S_{i\ell}=1} \exp(\text{score}_{i\ell})}, \qquad o_i = \sum_{j : S_{ij}=1} \alpha_{ij} v_j.$$

This is the clean semantic reference: all questions about correctness reduce to whether the dense mask implements the intended graph.

2.3 Schedule-based sparse HSA

The schedule-based sparse implementation is supposed to compute exactly the same function as the dense mask, but using explicit sparse descriptors rather than a dense masked traversal. In the repository, that compiler/runtime split lives primarily in:

- `flash_attn/cute/hsa.py`,
- `flash_attn/cute/hsa_cached_2d_forward_analysis.py`,
- `nanochat/hsa_batch_cache.py`.

The schedule is therefore a *compiled sparse realization of the same binary support graph*, not a different model class.

3 What the Current Graph Semantics Actually Encode

Current HSA is not general graph attention. It is a decoder-causal sparse graph with a specific information-routing policy derived from hierarchy.

In the present recipe, the graph includes:

- same-sentence body-level connectivity,

- same-section anchor connectivity,
- same-document anchor connectivity,
- cross-level edges such as body $\rightarrow$ previous sentence anchor,
- body $\rightarrow$ previous section anchor,
- sentence anchor $\rightarrow$ previous section anchor.

These cross-level edges matter. Without them, the graph is too lossy: a purely same-level tree-shaped routing policy under-connects the model and discards useful long-range semantic pathways.

The important subtlety is that this graph is a *routing graph*, not an ontology. The document hierarchy is a tree. Useful attention flow is closer to a causalized directed acyclic graph over the prefix.

4 Dense HSA Versus Sparse HSA

4.1 Dense HSA

Dense HSA means: build the exact semantic mask, run the ordinary dense masked attention reference, and let the kernel traverse all entries in the lower triangle even if many entries are masked out. This is conceptually simple and is the best semantic oracle when debugging graph meaning.

4.2 Sparse HSA

Sparse HSA means: compile the same semantic mask into sparse descriptors and realize the same attention via sparse traversal, sparse runtime metadata, or packed subproblems. Sparse HSA is therefore a systems optimization problem on top of the same graph semantics.

4.3 Invariant that matters

The critical invariant is:

$$\text{dense semantic mask} \equiv \text{sparse schedule} \equiv \text{kernel support}.$$

When these differ, training behavior diverges even if the intended high-level semantics were correct. Much of the engineering difficulty in HSA is therefore not inventing a graph, but proving that every layer of the implementation realizes the same graph.

5 Why Packing Helps Forward More Than Backward

5.1 Forward is naturally query-owned

Forward attention on a compressed subproblem is naturally row-owned:

$$Q_b \in \mathbb{R}^{r \times d}, \quad K_b, V_b \in \mathbb{R}^{u \times d},$$

with one output O_b and one log-sum-exp state per query row. Packing reduces an irregular sparse neighborhood into a regular dense microproblem.

That is why forward benefits strongly from packing:

- gather and pack once,
- compute one tiny dense exact attention,
- write one output and one LSE per row.

5.2 Backward has an ownership mismatch

Backward is harder because the natural ownership splits:

$$dQ = dSK \quad \text{(query-row owned)}$$

but

$$dK \mathrel{+}= dS^\top Q, \qquad dV \mathrel{+}= P^\top dO \quad \text{(key-row owned)}.$$

This means the same forward packing that makes a tiny dense forward problem does not automatically make a tiny dense backward problem with a natural writeback owner. The moment we leave the query-owned world, we pay for reductions, atomics, or extra saved-state movement.

5.3 Why `sparse_mask` backward is still so good

Even though `sparse_mask` backward is coarser and often less elegant than the most compressed forward path, it remains strong for several reasons:

1. it stays close to the exact support graph;
2. it amortizes work in large regular backward kernels instead of carrying fragile packed forward state into backward;
3. it avoids the q-owned / k-owned mismatch becoming a bespoke packed writeback problem;
4. once numerical issues are fixed, it is robust and easy to validate against the dense masked reference;
5. empirically, the forward gains from aggressive packing are large, while the backward gains from the same packing are usually small or negative.

Historically this is why the project leaned on compressed forward plus a more conservative backward. In the current codebase, however, the practical long-context training configuration is no longer "plain `sparse_mask`" as such; it is the packed HSA path that reuses cached forward sidecars and the dedicated long-context backward route attached to that path.

5.4 Long-context caveat: the generic block-sparse dQ path was the real bug

The long-context debugging pass went through two phases. The first phase correctly established that the dense `mask_mod` fallback backward is too slow for real long-context training. The second phase refined that diagnosis: the problem was not "fast backward" in the abstract, but the *generic* FA4 block-sparse dQ path used by the exact packed-mask traversal.

The key measurement trap was that the original isolated `fwdbwd` benchmark was profiling a degenerate backward regime. At step 0, the attention output projection `c_proj` is zero-initialized, so the upstream gradient flowing into the attention output is effectively zero. Because the implementation has an exact zero-`dout` fast path, that benchmark mostly measured:

- the real packed forward cost, plus
- an almost trivial backward.

Once we added a profiler mode that performs a real optimizer update before the profiled step, the true long-context picture became obvious. The dense-traversal fallback backward was indeed safe but far too slow, but the first fast replacement—the generic FA4 block-sparse backward used by `_run_hsa_packed_mask_backward(...)`—was still wrong.

The decisive tensor-level probe was a direct comparison on the exact same post-update 32768 tensors:

- dK and dV from the generic block-sparse traversal matched the dense-traversal oracle to numerical noise;
- dQ did not.

Typical differences were

$$\max |dQ_{\text{dense}} - dQ_{\text{sparse}}| \approx 5.5 \text{ to } 7.7, \qquad \text{mean}|dQ_{\text{dense}} - dQ_{\text{sparse}}| \approx 0.15 \text{ to } 0.18,$$

even though the upstream `dout` on that call was only of order 10^{-8}. That is not ordinary BF16 drift; it points to a wrong or stale dQ accumulation path.

5.5 What actually worked

The important refinement is that the *dedicated HSA long-context backward* paths already in the repository were not affected in the same way.

- Sparse exact HSA with `legacy_packed` mode uses the dedicated HSA long-context backward implementation.
- Cached generalized forward can now route its backward through the dedicated packed HSA backward as well.

On a real Gutenberg cache at $T = 32768$, first real post-update step:

- correctness-safe dense-traversal sparse path:

$$\text{loss}_1 = 6.4592276, \qquad \text{step}_1 \approx 22.2 \text{ s}$$

- dedicated HSA `legacy_packed` sparse path:

$$\text{loss}_1 = 6.4592276, \qquad \text{step}_1 \approx 10.2 \text{ s}$$

and the observed gradient drift stayed at BF16 scale rather than exploding in dQ.

On a fixed-seed resident five-step run at the same length:

- correctness-safe sparse: final loss ≈ 3.32278, throughput ≈ 1.37k tok/s;
- `legacy_packed` sparse: final loss ≈ 3.32252, throughput ≈ 6.64k tok/s;
- cached generalized forward plus dedicated packed HSA backward: final loss ≈ 3.32252, throughput ≈ 8.81k tok/s.

5.6 Current policy

The practical policy is therefore no longer "dense fallback everywhere until the long-context bug is fixed." It is:

1. keep the generic FA4 block-sparse backward opt-in only, because its dQ path is still not trustworthy at long context;
2. keep short contexts on the simple correctness-first path;
3. default long contexts to the dedicated HSA `legacy_packed` backward once sequence length crosses the auto threshold;
4. let cached generalized forward reuse that same dedicated packed HSA backward so the packed forward win survives end-to-end.

The current auto threshold is controlled by `FLASH_ATTN_HSA_AUTO_LEGACY_PACKED_BWD_MIN_SEQLEN` and defaults to 32768.

5.7 What changed in the implementation

The practical speedup came from routing changes, not from changing the HSA algebra. There are now three relevant backward families:

- dense traversal over the exact packed mask modifier;
- the generic FA4 block-sparse traversal driven by `block_sparse_tensors`;
- the dedicated HSA long-context backward implementations (`legacy_packed` and related HSA-specific paths).

The exact packed-mask helper still retains the conservative `block_sparse_tensors=None` route as the dense oracle, and the generic block-sparse traversal remains available for targeted kernel debugging. The important new routing change is that long-context HSA training no longer has to choose between "slow but correct dense traversal" and "fast but wrong generic block-sparse dQ." The repository now routes long contexts toward the dedicated HSA packed backward by default, while keeping the generic FA4 block-sparse path explicit and opt-in.

6 Row Packing, Column Packing, and Generalized 2-D Packing

6.1 Row packing

Row packing compresses multiple query rows with similar neighborhoods into a single regular microproblem. Informally:

- choose a small set of query rows,
- take the union of their live keys,
- gather that union once,
- run one tiny dense attention over the packed rows.

This is the most natural packing because forward attention is query-owned.

6.2 Column packing

Column packing focuses on the key/value side:

- compress repeated or shared key neighborhoods,
- reduce redundant K/V gathers,
- try to reuse union-K structure across rows.

This helps when many rows share similar key support, but on its own it is less complete than row packing because it does not resolve the query-side ownership pattern.

6.3 Generalized 2-D packing

Generalized 2-D packing treats the support matrix as a sparse binary matrix and tries to extract dense submatrices:

- exact dense tiles when a row group shares a common live key set,
- tiled residual tails for uncovered support,
- range-level fused execution where exact tiles and tails share one online softmax state.

That is more general than either pure row packing or pure column packing:

- row packing asks "which rows should be grouped?"
- column packing asks "which K/V neighborhoods should be reused?"
- generalized 2-D packing asks "which dense submatrices can be extracted from the sparse support matrix, and how should the remainder be tiled?"

The current cached generalized forward path in the repository is exactly this kind of 2-D extraction:

- exact dense core tiles,
- fused tiled tail,
- one range-local softmax state,
- one output write per row range.

6.4 What packed forward actually computes

The easiest way to say it precisely is: packing does *not* change the attention rule. It changes the layout of the live support so that dead "white-space" entries are physically removed before the kernel runs.

Take one sparse support subproblem with query rows $R = \{r_1, \dots, r_m\}$ and key rows $C = \{c_1, \dots, c_n\}$. Packing defines two gather maps

$$g_Q(a) = r_a, \qquad g_K(b) = c_b,$$

then gathers

$$\widetilde{Q}_a = Q_{g_Q(a)}, \qquad \widetilde{K}_b = K_{g_K(b)}, \qquad \widetilde{V}_b = V_{g_K(b)}.$$

It also defines the packed binary mask

$$\widetilde{M}_{ab} = \begin{cases} 1 & \text{if } (g_Q(a), g_K(b)) \text{ is live in the original HSA graph,} \\ 0 & \text{otherwise.} \end{cases}$$

Then the packed microproblem is just exact masked attention on the reindexed live rows and columns:

$$\widetilde{S}_{ab} = \frac{\widetilde{Q}_a^\top \widetilde{K}_b}{\sqrt{d}}, \qquad \widetilde{P}_{ab} = \frac{\exp(\widetilde{S}_{ab})\widetilde{M}_{ab}}{\sum_c \exp(\widetilde{S}_{ac})\widetilde{M}_{ac}}, \qquad \widetilde{O}_a = \sum_b \widetilde{P}_{ab}\widetilde{V}_b.$$

So the intuitive picture

> identify sparse tiles, remove the white space, fill the survivors into a dense tile, run GEMM, and then reindex so the softmax is correct

is basically right, with one important refinement: the softmax is not "corrected afterward" in an ad hoc way. The packed row/column indices and the packed mask $\widetilde{M}$ define the exact local normalization domain. When a single original HSA row is split across several packed families or ranges, the implementation carries an exact online softmax state (LSE / running max-sum) across those pieces and merges them before the final writeback. That is what keeps the packed forward algebraically identical to the original sparse row.

6.5 What packed backward actually computes

Backward uses the same gather maps, but the ownership is no longer as clean as in forward. In packed coordinates,

$$d\widetilde{Q} = d\widetilde{S}\,\widetilde{K}, \qquad d\widetilde{K} \mathrel{+}= d\widetilde{S}^\top \widetilde{Q}, \qquad d\widetilde{V} \mathrel{+}= \widetilde{P}^\top d\widetilde{O}.$$

The writeback is then:

- scatter $d\widetilde{Q}$ back to the original query rows $g_Q(a)$;
- scatter-add $d\widetilde{K}$ and $d\widetilde{V}$ back to the original key rows $g_K(b)$.

This is the core asymmetry:

- forward is naturally query-owned, so packing mostly gives a small exact dense microproblem plus one output write per row;
- backward must reduce many packed contributions back into shared original K/V rows, so the host-side descriptor path, batching policy, and scatter-add structure matter much more.

That is why "packing helps forward" does not automatically imply "the same packing gives an equally clean backward." The algebra is exact in both cases; the systems problem is harder in backward because the packed representation has to support cheap accumulation back into the original coordinates.

6.6 Operational picture: what the kernels are actually doing

The shortest faithful description is:

1. start from one sparse support tile in the original coordinates;
2. remove dead rows/columns and gather the surviving rows/columns into a smaller packed microproblem;
3. run exact masked attention on that packed microproblem;
4. scatter the output rows back, and in backward scatter $d\widetilde{Q}$ while scatter-adding $d\widetilde{K}, d\widetilde{V}$ into shared original key rows.

Figure 1 makes this concrete. The packed forward does *not* approximate the softmax. It evaluates the exact masked attention rule on a reindexed live submatrix, then maps the result back to the original rows.

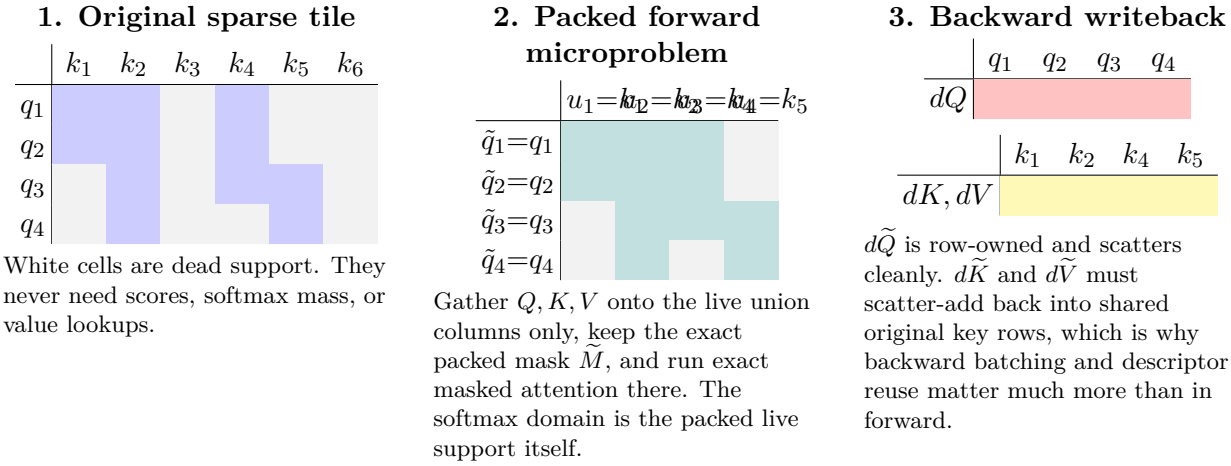
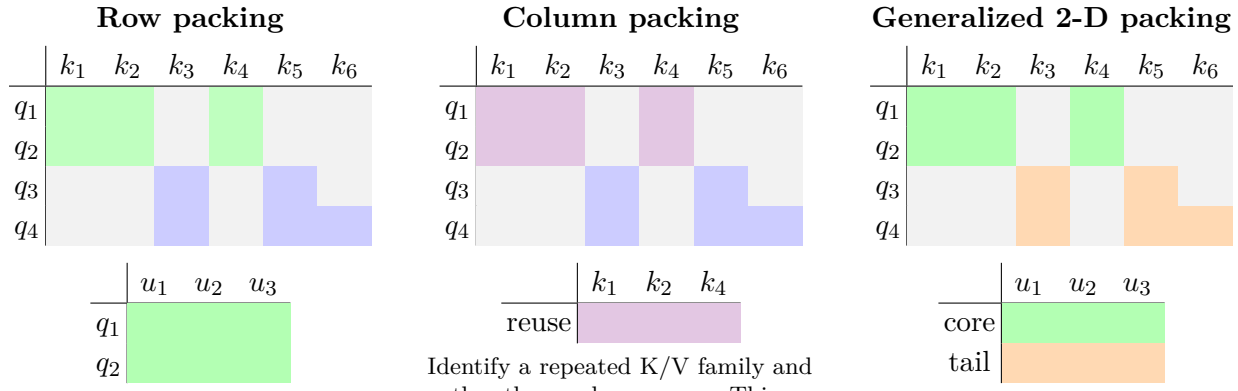


Figure 1: Operational picture of packed HSA. Forward removes dead support, reindexes the live rows/columns into a smaller exact masked microproblem, and scatters the result back. Backward uses the same gather maps, but K/V are shared across rows so the writeback is a reduction problem rather than a simple row write.

6.7 Packing illustrations

Figure 2 shows the three packing viewpoints on a small support matrix. Blue cells are live edges in the original sparse support. Green cells indicate an extracted dense core, purple cells indicate a reused column family, and orange cells indicate a tiled residual tail.

The important systems point is that all three methods operate on the support graph rather than on the exact scoring rule. This means the graph compiler, sidecars, schedule families, and most of the current packing machinery remain reusable if the score becomes relation-aware or typed. The part that changes is the kernel math inside each packed family: score evaluation, value transforms, and possibly normalization if the model moves beyond plain masked softmax.



Select similar rows $\{q_1, q_2\}$, form the union $\{k_1, k_2, k_4\}$ once, and run one dense 2×3 microproblem.

Identify a repeated K/V family and gather those columns once. This reduces redundant K/V movement, but row ownership is still unresolved by itself.

Extract dense submatrices from the support itself: an exact dense core tile for (q_1, q_2), then cover the remainder with tiled residual tails that share one range-local softmax state.

Figure 2: Three views of packing. In every case the blue/green/orange cells are *live* support and the white cells are dead support that is physically removed from the packed microproblem. Row packing groups similar query rows, column packing reuses repeated K/V families, and generalized 2-D packing extracts dense submatrices from the support matrix and covers the remainder with fused tail tiles. The packed microproblem is still exact masked attention on the reindexed live rows and columns, and online softmax/LSE merging keeps normalization identical to the original sparse row even when one row is split across several packed ranges.

7 Benchmark Baselines and Preliminary Results

7.1 Benchmark labels

The current benchmark story is easiest to understand if the runtime modes are named by *what work is actually being done*:

- **flat FA4**: ordinary dense causal FA4 with no hierarchical sparsity;
- **sliding_log**: a causal sliding-window FA4 baseline with window width $\lceil \log_2 T \rceil$;
- **strict_log**: the same logarithmic sliding-window width $\lceil \log_2 T \rceil$ applied at *every* layer, including the final layer;
- **sliding_fixed**: a causal sliding-window FA4 baseline with a fixed window width of 1000 tokens;
- **HSA naive**: exact HSA with cached-forward fallback disabled, i.e. the adjacency-driven sparse realization;
- **HSA packed**: exact HSA with cached-forward fallback enabled, i.e. the packed HSA path that reuses precomputed forward sidecars and the corresponding long-context backward route.

This distinction matters because once runtime fallback to precomputed cached sidecars is enabled, a plain `sparse_mask` benchmark is no longer a clean "naive sparse" baseline on cache-backed batches. In current cache-backed runs, the default exact HSA path can already reuse the packed cached forward representation. For that reason, the meaningful systems comparison is now **HSA naive** versus **HSA packed**, not `sparse_mask` versus an older explicit cached preset.

The distinction between **sliding_log** and **strict_log** is operationally important. **sliding_log** is not a pure local baseline; it is a logarithmic local stack plus one dense global final layer. By contrast, **strict_log** is a pure log-band local stack. Recent Gutenberg, FineWeb, and Wikipedia slices suggest that both are much stronger than a toy sparse strawman and should remain in the benchmark set.

7.2 Isolated packed-versus-naive HSA

Table 1 gives the clean isolated comparison on cache-backed resident batches. These are the same HSA semantics, same cached batches, same model, and same losses to numerical noise; the only difference is whether the runtime is forced onto the naive adjacency-driven sparse path or is allowed to consume the packed cached sidecars.

T	naive dt (ms)	packed dt (ms)	packed tok/s	speedup	loss match
65536	6549.4	118.2	554.6k	55.4×	yes
131072	15657.0	207.5	631.6k	75.4×	yes
262144	29184.6	344.7	760.6k	84.7×	yes

Table 1: Isolated HSA systems comparison on cache-backed resident batches. The packed path is the intended method; the naive path is the adjacency-driven reference that does not reuse packed cached sidecars.

The key result is not subtle: the packed path is already doing the intended systems job. The speedup over the naive exact sparse realization grows with sequence length, from roughly 55× at 64K to roughly 85× at 262144, while preserving the same hierarchical support semantics.

7.3 True multi-batch resident training runs

The earlier repeated-single-batch resident smokes were useful for quick systems checks, but they were too degenerate to say much about relative learning behavior. Table 2 replaces that with *true multi-batch* resident training on a scaled Gutenberg corpus (`gutenberg_large`, 31 books, ≈ 23.6M characters, and real long-context train-cache budgets of 227 batches at 64K, 116 at 128K, and 59 at 262144). To keep the resident benchmark memory-bounded while still avoiding repeated-single-batch memorization, the runs below cycle over selected resident subsets:

- 64K: 8 distinct resident train batches;
- 128K: 3 distinct resident train batches;
- 262144: 2 distinct resident train batches.

All rows below use real cache-backed batches and cycle over multiple distinct resident schedules rather than memorizing a single cached example.

Several points matter immediately.

1. On the scaled corpus, the packed HSA path is already clearly ahead of dense flat FA4 by 64K on both throughput and loss, and by 128K and 262144 the throughput gap is large: about 5.3× at 128K and about 5.0× at 262144, while also landing at materially lower loss on these short internal runs.

T	mode	train batches	final loss	mean dt (ms)	tok/s
65536	flat FA4	8	2.9195	299.7	218.7k
65536	sliding_log	8	2.3961	65.4	1001.8k
65536	sliding_fixed	8	2.8806	60.3	1087.0k
65536	HSA packed	8	2.4858	130.2	503.3k
131072	flat FA4	3	2.8504	947.2	138.4k
131072	sliding_log	3	1.7216	199.6	656.7k
131072	sliding_fixed	3	2.8088	306.2	428.1k
131072	HSA packed	3	1.8358	178.8	733.2k
262144	flat FA4	2	2.8278	1740.8	150.6k
262144	sliding_log	2	1.3794	358.4	731.3k
262144	sliding_fixed	2	2.6345	367.8	712.7k
262144	HSA packed	2	1.4889	351.6	745.6k

Table 2: True multi-batch resident training runs on the scaled `gutenberg_large` long-context caches. These are still internal benchmark runs rather than full downstream training jobs, but they are materially stronger than repeated-single-batch smokes because they cycle over multiple distinct resident schedules from a larger real corpus.

2. The sliding-window baselines are much stronger than a toy strawman. They are not hierarchical and they should eventually lose on relevance when the task genuinely requires nonlocal graph structure, but they are very fast and must be included in any honest benchmark table.

3. On these short true multi-batch resident runs, `sliding_log` remains the strongest quality baseline. That is a useful warning: any claim that HSA "obviously" dominates sliding windows needs to be supported by longer and less degenerate training, not by intuition alone.

4. The earlier scaled-corpus 64K anomaly was not a missing packed dispatch. It came from a bad interaction between first-use per-schedule costs and a subset of packed sidecars that chose grouped `k_window`/direct families instead of the fast `union_2d` family. Rebuilding the selected 64K sidecars with a benchmark-only policy override that forces the efficient unionized family, and warming all selected resident batches before timing, restores the expected packed-HSA regime.

The 262144 row remains the clearest systems crossover. Packed HSA cleanly clears the dense flat baseline on both throughput and loss while landing slightly ahead of both sliding baselines on throughput, though still behind `sliding_log` on loss. That still does not prove a general downstream quality win by itself, but it is the right systems crossover: the packed hierarchical path is now winning in the regime it was designed for, and the remaining work is to run longer multi-batch training on larger corpora and to improve smaller long-context regimes without relying on benchmark-specific sidecar retuning.

7.4 Issues encountered after the Gutenberg crossover

The Gutenberg table established that packed HSA can already cross over dense flat FA4 in the long-context regime. The next debugging pass on Wikipedia and FineWeb exposed a different issue: the packed representation is dataset-geometry-sensitive.

The main problems were:

1. some wiki/web batches produced packed sidecars with large grouped `k_window` / direct families rather than the compact `union_2d` regime that worked well on Gutenberg;
2. those sidecars often lacked a cheap cached fused-backward route, so the runtime fell back to the legacy packed backward machinery;
3. the remaining runtime bottleneck on those batches was not the dense GEMM itself, but host-side descriptor building and thousands of tiny packed backward panel calls.

One concrete improvement already landed early in this pass: sorting and coalescing legacy packed backward panels by shape before execution. On a representative Wikipedia 64K batch, that reduced the hybrid backward batch count from roughly

$$165 \longrightarrow 15$$

under the current default merge budget, and reduced the first real post-update step from about

$$9.17 \text{ s} \longrightarrow 1.65 \text{ s}.$$

That is an important refinement of the current state:

- the remaining non-Gutenberg blocker is not "HSA semantics are wrong";
- it is also not "GEMM packing is inherently slower than FA4";
- it is a systems issue in the packed backward descriptor/batching path on less favorable support geometries.

So the current non-Gutenberg work is about making the packed backward path less fragile to wiki/web geometry, not about changing the HSA graph definition.

The later fixes were more important still:

1. regroup `direct_passthrough` families across qgroups instead of respecting qgroup boundaries too aggressively;
2. regroup `k_window` families across qgroups in the same way;
3. add a real cached backward fast path for `masked_union` payloads rather than falling back to legacy packed backward;
4. increase the cached masked-forward range chunk size so very large `k_window` ranges are not executed as dozens of tiny masked-kernel launches.

The effect of the masked-union cached backward path is easy to see on single batches:

case	old backward	new backward
Wikipedia 64K batch 0	250.2 ms	34.3 ms
FineWeb 128K batch 0	586.6 ms	106.0 ms

That eliminated the old backward-plumbing bottleneck. After that point, the remaining wiki/web cost was mostly in packed forward geometry, especially on large `k_window` ranges.

7.5 What the wiki/web geometry actually says

Once the backward setup path was fixed, the next blocker was no longer generic host-side sparse plumbing. It was the *shape* of the packed problem on wiki/web batches.

On a representative Wikipedia 64K batch, the cached packed payload was:

- total groups: 8008,
- `direct_passthrough` groups: 7756,
- `k_window` groups: 28,
- `union_2d` groups: 224.

The more important number is the implied hardware area. On that same batch, the direct family alone accounted for roughly

$$881{,}966/896{,}751 \approx 98.3\%$$

of the packed hardware area. So the practical problem was not "a few awkward tails." The practical problem was that the payload had been fragmented into thousands of tiny direct groups.

This led to the current planner insight:

> for these wiki/web batches, it is better to accept a slightly larger packed union if it materially reduces group count and launch count.

That is visible in the direct-family regroup experiment on the same Wikipedia 64K batch:

quantity	old payload	regrouped direct payload
direct group count	7756	995
total group count	8008	1247
direct hardware area	881,966	1,040,390
packed forward time	5.46 ms	2.24 ms

So the direct regrouping increases direct packed area by only about 18%, but reduces packed forward time by about 2.44×. That is the missing systems lesson: on non-Gutenberg geometry we were over-optimizing for local packing density and under-optimizing for group granularity.

The same pattern reappeared once direct regrouping was fixed. On Wikipedia 64K, batches 2 through 4 were still dominated by huge `k_window` ranges. After the `k_window` regrouping fix and the masked-union cached backward path, those batches were no longer backward-dominated; they were forward-dominated. The next winning change was to raise the cached masked-forward range chunk size from 128 groups to 512. On representative slow wiki batches this reduced forward time as follows:

batch	chunk 128	chunk 512
Wikipedia 64K batch 2	149.0 ms	103.3 ms
Wikipedia 64K batch 4	154.3 ms	112.5 ms
FineWeb 128K batch 0	160.9 ms	109.8 ms

So the final non-Gutenberg crossover was not one bug. It was a sequence:

1. reduce direct fragmentation,

2. reduce `k_window` fragmentation,

3. replace legacy packed backward with cached masked-union backward,

4. then reduce forward launch overhead on the remaining large masked ranges.

With those fixes in place, the representative non-Gutenberg resident runs now look like this:

dataset	mode	mean dt	tok/s
Wikipedia 64K	flat FA4	143.5 ms	456.8k
Wikipedia 64K	HSA packed	128.8 ms	509.0k
FineWeb 128K	flat FA4	476.2 ms	275.2k
FineWeb 128K	HSA packed	188.3 ms	696.2k

That is the first point where the cross-dataset story becomes coherent: Wikipedia 64K now clears flat FA4, and FineWeb 128K is well into the regime where packed HSA is materially faster than flat.

7.6 The apparent non-convergence issue was a benchmark LR mismatch

One later scaled benchmark pass initially suggested that packed HSA was not converging on Wikipedia and FineWeb, even though the step times were already in the right systems regime. That turned out to be a harness issue, not an HSA correctness failure: the hierarchical rows in that pass had been run with `-hierarchical-lr-scale 0.1`, so the packed model was effectively trained at one tenth of the baseline learning rate.

After rerunning the same resident benchmarks at `hierarchical_lr_scale`=1.0, the packed rows returned to the same loss regime as flat FA4:

dataset	mode	lr scale	final loss	tok/s
Wikipedia 131072	flat FA4	1.0	3.3347	276.2k
Wikipedia 131072	HSA packed	1.0	3.3210	854.9k
Wikipedia 131072	strict_log	1.0	3.3208	1.527M
FineWeb 262144	flat FA4	1.0	3.3236	151.1k
FineWeb 262144	HSA packed	1.0	3.3011	692.5k
FineWeb 262144	strict_log	1.0	3.2968	1.586M

So the corrected conclusion is narrower and cleaner:

- packed HSA does converge comparably to flat FA4 on these scaled wiki/web runs when trained at the same learning rate,
- the earlier poor packed-loss rows were an unfair benchmark setting, not evidence of a masked-union backward correctness problem,
- and the remaining competitive problem is the strength of **strict_log**, not a flat-versus-packed convergence failure.

7.7 Which knobs are actually worth tuning

The current tuning picture is now clearer:

- `max_rows_per_group`: the main granularity knob. Larger groups amortize launch and descriptor cost, but can increase union area. This is the first knob to push once fragmentation is the dominant problem.
- `max_union_k_direct`: the ceiling for direct-family unions. If it is too small, direct work fragments; if it is too large, direct groups bloat and lose their advantage.
- `direct_density_threshold`: a secondary family-assignment knob. On the current Wikipedia 64K slice, lowering it from 0.85 to 0.55 only moved runtime by about 1.1%, so it is not the dominant lever.
- `window_density_threshold` and `k_gap_threshold`: route work between `direct_passthrough`, `k_window`, and `union_2d`. They matter when families are misclassified, but they do not fix fragmentation by themselves.
- `packed_forward_block_q`: q-side granularity. On the current Wikipedia 64K benchmark, 256 outperformed 128.
- `tile_k`: kernel tile width. This matters only after the group shapes are in a reasonable regime; it is not the first lever to pull when the payload is still exploding into tiny groups.
- `FLASH_ATTN_HSA_CACHED_MASKED_GROUP_CHUNK`: runtime chunking for large masked ranges. Once the payload has already been regrouped, this becomes the main forward-side launch-amortization knob for very large `k_window` ranges. On the current wiki/web slices, 512 was clearly better than 128, while 1024 did not improve further.

So the practical tuning order is:

1. reduce group fragmentation,
2. then fix the backward path for the dominant residual family,
3. then tune masked-range runtime chunking,
4. only then tune direct-versus-union family assignment and tile-level kernel parameters.

7.8 MFU caveat

The benchmark harness currently reports MFU using a dense-equivalent FLOP estimate. That is serviceable for flat and sliding-window FA4 rows, but it is not publishable for HSA rows because the packed and sparse paths do not execute the dense-equivalent workload. For now, the trustworthy systems metrics are:

- mean step time,
- tokens per second,
- peak memory,
- and loss trajectories.

8 What the Current Implementation Already Gives Us

The current stack contains several reusable components that are more general than "hierarchy only" suggests.

8.1 1. A graph compiler from token annotations to sparse causal support

Today this compiler is expressed through:

- `keep_ids`,
- `hash_ids`,
- `compute_hsa_mask(...)`,
- `build_hsa_schedule(...)`.

This is already a graph compiler. It just currently compiles one specific family of hierarchical binary support graphs.

8.2 2. Exact dense and sparse references

The repository already has the conceptual machinery needed to separate:

- intended semantics,
- sparse schedule,
- kernel behavior.

That is crucial for any richer graph scheme. If the graph is extended to typed or directed edges, the same discipline will still be necessary.

8.3 3. Sidecars and offline precomputation

The schedule sidecar and forward sidecar system is not specific to hierarchy. It is a reusable vehicle for:

- offline graph compilation,
- offline packing,
- runtime attachment of structural metadata,
- avoiding host-side rebuild costs.

This becomes even more valuable if the graph moves from simple hierarchy to typed semantic edges, since graph compilation will likely become more expensive.

8.4 4. Packing policy and schedule families

The current row-compact and generalized 2-D packing framework is a reusable "graph-to-kernel-family" compiler. The main idea is already in place:

- start from an irregular support graph,
- partition it into a small set of regular families,
- run specialized kernels on each family.

That is exactly the right systems pattern for richer semantic graphs too.

9 What Does *Not* Recycle Directly

Not every part of the current HSA stack is directly reusable for a leap toward semantic or causal graphs.

9.1 1. Binary support is too weak for typed semantics

If all edges are reduced to a single binary mask, then

$$\texttt{Spain} \to \texttt{Europe} \quad \text{and} \quad \texttt{rain} \to \texttt{wet}$$

look identical at the routing level. The graph topology survives, but relation type does not.

9.2 2. Standard softmax attention only models positive mixing weights

Softmax produces

$$\alpha_{ij} \geq 0, \qquad \sum_j \alpha_{ij} = 1.$$

This is appropriate for compatibility-based readout, but it is not a natural model of signed causal influence. The output can still become negative through the value vectors, but the weighting mechanism itself is not signed.

9.3 3. Current kernels assume binary support plus standard reductions

The current sparse kernels assume:

- allowed or forbidden edges,
- standard score computation,
- standard softmax normalization,
- standard $QK^\top$ and weighted V reductions.

Typed, signed, or relation-conditioned message passing would require either:

- multiple edge-family kernels,
- relation-specific score transforms,
- relation-specific value transforms,
- or an alternative aggregator that is no longer a pure masked softmax.

10 From Hierarchy to Typed Directed Semantic Graphs

10.1 The right abstraction

The natural extension is not "arbitrary graph attention" in the abstract. The natural extension is:

> decoder-causal sparse routing over tokens and semantic memory nodes, with typed directed edges.

This still respects autoregressive training, but allows the graph to encode more than document boundaries.

10.2 Semantic direction versus attention direction

There are two distinct notions of direction:

1. **semantic direction:** e.g. a knowledge or event relation,
2. **attention-flow direction:** which previously visible node a token is allowed to read from.

These are not the same thing. A semantic graph may contain a relation

$$\texttt{rain} \to \texttt{wet},$$

but in a decoder we still must causalize it into past-visible reads only. So a semantic graph must be compiled into a causal routing graph before it becomes an attention support.

10.3 A minimal typed extension

The smallest meaningful extension from current HSA is:

- keep binary support families,
- add edge type τ_{ij},
- modify the score and/or transmitted message by type.

A minimal relation-aware score is:

$$\text{score}_{ij} = q_i^\top W_{\tau_{ij}} k_j + b_{\tau_{ij}}.$$

A minimal relation-aware message is:

$$m_{ij} = \alpha_{ij}\, T_{\tau_{ij}}(v_j), \qquad o_i = \sum_j m_{ij}.$$

This already gives a typed directed semantic graph while remaining relatively close to the current masked-attention implementation model.

11 Toward Signed and Causal Message Passing

11.1 Why signed edges are interesting

Some relations are naturally excitatory while others are inhibitory. For example, a world-state graph might treat

- `rain` → `wet-ground` as positive,
- `watering` → `wet-ground` as positive,
- `sun/heat` → `wet-ground` as negative or drying.

A pure binary mask cannot distinguish these. Even a typed score only changes who is read from, not whether the message should reinforce or inhibit the target.

11.2 What softmax cannot express directly

Softmax gives positive simplex weights. So if the goal is to model signed or inhibitory influence directly, one likely needs one of:

1. relation-specific value transforms with sign,
2. dual positive/negative channels,
3. a non-simplex aggregator,
4. latent semantic state nodes where inhibitory structure is carried in state rather than in the raw attention weights.

11.3 A realistic near-term precursor

The most practical precursor to causal reasoning is not true interventional causality. It is:

> typed directed sparse message passing over tokens and semantic memory nodes, where the graph encodes permitted information pathways and relation-specific message transforms.

That is already a meaningful leap beyond hierarchy, while remaining compatible with autoregressive training.

12 Why This Is Not Yet True Causal Reasoning

A directed semantic graph inside a next-token predictor is still an *inductive bias*, not a causal model in Pearl's sense. True causal claims require more than directional edges:

- intervention semantics,
- counterfactual structure,
- assumptions about latent confounding,
- or explicit world-state modeling.

So the better description is:

> the current HSA machinery can be extended into a typed semantic routing framework that may support precursors to causal reasoning, but not causal inference merely by virtue of using a directed graph.

13 A Concrete Leapfrog Plan

13.1 Stage 1: typed sparse graph

Extend the current binary support into a small set of typed edge families:

- local,
- summary/hierarchy,
- entity-link,
- discourse,
- causal-positive,
- causal-negative.

This step can reuse:

- token-side metadata extraction,
- graph compilation,
- sidecars,
- packed family scheduling.

13.2 Stage 2: relation-aware score and message transforms

Keep the sparse routing framework, but make score and message depend on edge type. This is still close enough to current HSA that the implementation can evolve incrementally.

13.3 Stage 3: semantic memory nodes

Introduce explicit non-token nodes such as:

- entity nodes,
- event nodes,
- state nodes,
- section or chapter summaries,
- retrieved knowledge nodes.

Then sparse attention becomes routing between tokens and memory/state nodes, rather than only between raw tokens.

13.4 Stage 4: signed or non-simplex aggregation

Only after typed graph routing is stable should the scoring rule itself be generalized beyond pure masked softmax, for example by allowing signed message coefficients or separate positive/negative channels.

14 Summary

The current HSA implementation should be understood as *sparse causal routing*, not arbitrary graph attention. Its graph compiler, exact dense/sparse references, sidecar system, and packing framework already provide reusable infrastructure for a richer semantic-graph program.

The key engineering lessons so far are:

1. correctness depends on semantic mask, sparse schedule, and kernel support all matching exactly;
2. forward benefits strongly from aggressive packing because the problem is query-owned and easy to regularize;
3. packed training still uses the exact sparse backward, but long-context runtime now depends critically on choosing the block-sparse traversal instead of the dense fallback;
4. generalized 2-D packing is the right forward abstraction because it extracts dense submatrices from the support matrix itself;
5. the natural next leap is typed directed semantic routing, not a jump straight to unrestricted signed graph inference.

In short: the current method will not work with *any* graph and token scheme, but it is already close to a general graph compiler for *causalized sparse semantic routing*. That is a strong base for the next stage.

References

References

[1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. *Attention Is All You Need.* NeurIPS, 2017. https://arxiv.org/abs/1706.03762

[2] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Re. *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness.* NeurIPS, 2022. https://arxiv.org/abs/2205.14135

[3] Tri Dao. *FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning.* 2023. https://arxiv.org/abs/2307.08691

[4] Iz Beltagy, Matthew E. Peters, and Arman Cohan. *Longformer: The Long-Document Transformer.* 2020. https://arxiv.org/abs/2004.05150

[5] Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. *Big Bird: Transformers for Longer Sequences.* NeurIPS, 2020. https://arxiv.org/abs/2007.14062

[6] Petar Velickovic, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. *Graph Attention Networks*. ICLR, 2018. https://arxiv.org/abs/1710.10903

[7] Michael Schlichtkrull, Thomas N. Kipf, Peter Bloem, Rianne van den Berg, Ivan Titov, and Max Welling. *Modeling Relational Data with Graph Convolutional Networks*. ESWC, 2018. https://arxiv.org/abs/1703.06103

[8] Peter W. Battaglia, Jessica B. Hamrick, Victor Bapst, Alvaro Sanchez-Gonzalez, Vinicius Zambaldi, Mateusz Malinowski, Andrea Tacchetti, David Raposo, Adam Santoro, Ryan Faulkner, Caglar Gulcehre, H. Francis Song, Andrew J. Ballard, Justin Gilmer, George E. Dahl, Ashish Vaswani, Kelsey R. Allen, Charles Nash, Victoria Langston, Chris Dyer, Nicolas Heess, Daan Wierstra, Pushmeet Kohli, Matthew Botvinick, Oriol Vinyals, Yujia Li, and Razvan Pascanu. *Relational Inductive Biases, Deep Learning, and Graph Networks*. 2018. https://arxiv.org/abs/1806.01261

[9] Ziniu Hu, Yuxiao Dong, Kuansan Wang, and Yizhou Sun. *Heterogeneous Graph Transformer*. WWW, 2020. https://arxiv.org/abs/2003.01332

[10] Tyler Derr, Yao Ma, and Jiliang Tang. *Signed Graph Convolutional Network*. ICDM, 2018. https://arxiv.org/abs/1808.06354

[11] Judea Pearl. *Causality: Models, Reasoning, and Inference*. Cambridge University Press, second edition, 2009. https://www.cambridge.org/core/books/causality/B0046844FAE10CBF274D4ACBDAEB5F5B